Ontology-Schema Joint Retrieval for Data Agents: Bridging the Semantic Gap in Natural Language to SQL Generation

Anonymous Author
Anonymous Institution
USA
anonymous@email.com

Abstract

Data Agents that translate natural language queries into SQL face a fundamental challenge: the semantic gap between user language and database schema. Traditional approaches rely solely on database schema information, lacking the business domain knowledge necessary to understand complex queries involving business concepts, rules, and domain-specific terminology. This paper presents a novel Ontology-Schema Joint Retrieval (OSJR) framework that unifies business ontology and database schema into a heterogeneous knowledge graph, enabling multi-source retrieval that combines semantic, structural, and rule-based signals. Our approach employs a heterogeneous graph neural network (HeteroSchemaGNN) to learn joint embeddings, enabling cross-modal retrieval between business concepts and database elements. We further introduce an adaptive fusion mechanism that dynamically weights different retrieval signals based on query type. Extensive experiments on Spider and BIRD benchmarks demonstrate that our method significantly outperforms state-of-the-art baselines, achieving 5-12% improvement in recall and 7-15% improvement in SQL execution accuracy. The proposed framework provides a principled approach to bridging the semantic gap in Data Agent systems.

Keywords

Data Agent, Text-to-SQL, Ontology, Schema Linking, Joint Retrieval, Heterogeneous Graph Neural Network

1 Introduction

Data Agents, a class of AI systems capable of understanding natural language queries and autonomously accessing structured data systems, have become increasingly important in enterprise data management. These agents enable non-technical users to query databases using natural language, democratizing data access across organizations. From business intelligence dashboards to conversational interfaces, Data Agents are transforming how organizations interact with their data assets. The core challenge in building effective Data Agents is bridging the *semantic gap* between user queries expressed in natural language and the technical structure of databases (schema).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

The emergence of large language models (LLMs) has significantly advanced the capabilities of Data Agents. Modern systems can leverage extensive world knowledge and sophisticated reasoning abilities to understand complex user intents. However, even the most capable LLMs face fundamental limitations when translating natural language queries into accurate SQL statements. The primary bottleneck lies not in the generation capability, but in the retrieval and understanding of the relevant database context.

The semantic gap manifests in multiple dimensions, each presenting unique challenges for Data Agent systems:

- **Lexical Gap**: Users often employ business terminology that differs significantly from database column names. A user might ask for "high-value customers" while the database contains a column named "total_order_amount" or "customer_tier". This lexical mismatch can exceed 70% in enterprise databases where technical naming conventions diverge from business vocabulary.
- **Semantic Gap**: Business rules and calculations are often implicit in user queries. "Return rate" requires understanding that it equals returns divided by total orders; "revenue growth" requires knowing the formula for year-over-year comparison. Without explicit business knowledge, systems cannot correctly interpret these concepts.
- **Structural Gap**: Relationships between entities may span multiple tables through foreign key joins that are not apparent from the query alone. Users think in terms of business entities, not foreign key paths. A query like "orders with shipping details" requires understanding the join path: orders → order_items → products → inventory.
- **Temporal Gap**: Business concepts like "fiscal year", "quarter", "rolling 12 months", or "year-to-date" require temporal reasoning that goes beyond simple date filtering. Different organizations define these periods differently, and business knowledge is essential for correct interpretation.
- **Aggregation Gap**: Business metrics often involve complex aggregations. "Average customer lifetime value" requires understanding that it involves summing all purchases per customer, then averaging across customers. Similar confusion exists for "month-over-month growth", "year-over-year comparison", and "percentile rankings".

These gaps cause existing Text-to-SQL systems to fail on complex analytical queries, where business domain knowledge is essential for correct interpretation. Our analysis of failure cases in the Spider benchmark reveals that over 65% of errors stem from inadequate schema linking rather than SQL generation issues.

Existing approaches to Text-to-SQL generation primarily focus on schema linking, which identifies relevant database tables

and columns for a given query. However, these methods operate solely on database schema, ignoring the rich business knowledge that organizations maintain in ontologies, business glossaries, and domain-specific rules. This limitation becomes particularly acute for complex analytical queries that require business domain understanding. Organizations invest significant resources in documenting business concepts, defining key metrics, and establishing data dictionaries—resources that remain untapped in current systems.

In this paper, we propose Ontology-Schema Joint Retrieval (OSJR), a novel framework that unifies business ontology and database schema into a coherent representation, enabling Data Agents to leverage both technical database structure and business domain knowledge. Our approach treats the ontology and schema as complementary knowledge sources, jointly retrieving relevant elements from both to provide comprehensive context for SQL generation.

Our key contributions include:

(1) **Unified Knowledge Representation**: We introduce a heterogeneous knowledge graph that integrates ontology entities, database tables, and columns, along with alignment relations connecting business concepts to technical schema elements. This representation enables cross-modal reasoning between business semantics and database structure.
(2) **HeteroSchemaGNN Encoder**: We develop a heterogeneous graph neural network that learns joint embeddings for all node types in the unified knowledge graph, enabling semantic alignment between ontology and schema representations through contrastive learning.
(3) **Multi-Source Retrieval**: We develop a retrieval system that combines semantic similarity, structural connectivity through foreign key relationships, and rule-based matching to comprehensively identify relevant knowledge for a given query.
(4) **Adaptive Fusion**: We propose an adaptive fusion mechanism that dynamically adjusts the weights of different retrieval signals based on query characteristics, optimizing performance across different query types.
(5) **Comprehensive Evaluation**: We conduct extensive experiments on Spider and BIRD benchmarks, demonstrating significant improvements over state-of-the-art baselines, including 5-12% improvement in recall and 7-15% improvement in SQL execution accuracy.

Empirically, we show that our approach achieves state-of-the-art results on both benchmarks. On Spider, we achieve 89% execution accuracy, a 7% absolute improvement over the previous best. On BIRD, we achieve 73% accuracy on hard queries, an 8% improvement. These results validate our hypothesis that business domain knowledge is essential for complex analytical queries.

To summarize, our work makes the following key observations: (1) business domain knowledge is essential for understanding complex analytical queries but is missing from existing Text-to-SQL systems, (2) business ontologies provide a natural representation for this domain knowledge, (3) integrating ontologies with database schema enables cross-modal retrieval that leverages both sources, and (4) adaptive fusion based on query type optimizes retrieval performance across different query patterns.

We further highlight the practical significance of our approach. In enterprise environments, Data Agents face a fundamental challenge: bridging the gap between business users who think in terms of business concepts and technical databases that store data in normalized schemas. Our approach directly addresses this challenge by leveraging the business ontology that enterprises already maintain. This means organizations do not need to build new knowledge bases; they can simply integrate their existing business glossaries, data dictionaries, and domain ontologies into our framework.

The rest of this paper is organized as follows. Section 2 discusses related work on Text-to-SQL and schema linking. Section 3 provides necessary background and defines key concepts. Section 4 presents our methodology in detail. Section 5 describes the experimental setup, and Section 6 presents the results. Section 7 concludes the paper.

2 Related Work

Text-to-SQL generation has evolved through three main phases: rule-based methods, neural sequence-to-sequence models, and large language model (LLM) based approaches [20, 22]. We review each phase and discuss their limitations in handling business domain knowledge.

2.1 Text-to-SQL Generation

Early rule-based systems like LUNAR [38] used grammar-based rules to map natural language to SQL queries. These systems relied on manual rule construction and lacked the ability to generalize to new domains or query patterns. The maintenance burden of these systems became unsustainable as database complexity grew.

The neural era saw the development of encoder-decoder architectures for Text-to-SQL. Models like Seq2SQL [42] introduced reinforcement learning for SQL clause ordering. IRNet [8] proposed a semantic grammar-based approach that first predicts an intermediate representation before generating SQL. RAT-SQL [34] introduced relation-aware transformers that explicitly model table relationships through attention biasing, achieving state-of-the-art performance on Spider. The key innovation of RAT-SQL was treating table relationships as explicit edges in the attention mechanism, enabling the model to better capture foreign key relationships.

More recent neural approaches have explored diverse architectures. GraphSQL used graph-structured attention to model table relationships. SQLNet introduced column attention to better identify relevant columns. SMBoP [19] proposed a bottom-up parsing approach that builds SQL queries incrementally. S2SQL [11] incorporated syntax information into the question-schema interaction graph encoder. Each of these methods improved over their predecessors but remained limited to schema-only context.

The emergence of large language models (LLMs) [2, 24, 32] marked a paradigm shift in Text-to-SQL research. Rather than training models from scratch, LLMs can be prompted with few-shot examples to generate SQL. This approach leverages the extensive knowledge encoded in pre-trained models, reducing the need for task-specific training data. However, as discussed earlier, even powerful LLMs struggle with queries requiring business domain knowledge not present in the database schema.

Recent advances in large language models have enabled zero-shot and few-shot Text-to-SQL. DIN-SQL [27] decomposed the SQL generation process into sub-tasks with dedicated prompts for schema linking, column selection, SQL generation, and SQL ordering. DAIL-SQL [7] introduced diversity-augmented in-context learning with execution-guided decoding, sampling examples that maximize schema coverage. More recent work like CHASE-SQL explored chain-of-thought prompting for complex queries. Despite these advances, existing LLM-based methods still struggle with complex queries requiring domain knowledge that is not present in the database schema.

The fundamental limitation shared by all these approaches is their exclusive reliance on database schema for context. They cannot leverage the business knowledge that organizations maintain in ontologies, glossaries, and domain-specific rules.

2.2 Schema Linking

Schema linking, the process of identifying relevant database tables and columns for a given query, is widely recognized as a critical bottleneck in Text-to-SQL systems [16]. Early approaches relied on exact string matching or TF-IDF keyword similarity between query tokens and schema element names. These methods failed to handle semantically equivalent but lexically different terms, such as matching "customer" to "client_id".

Modern schema linking methods employ semantic embeddings computed by pre-trained language models [5]. BERT-based encoders [30] compute contextualized representations of schema elements and queries, enabling fuzzy matching between dissimilar terms. Graph neural networks [15, 33] capture structural relationships between tables through foreign keys, enabling multi-hop reasoning over the schema. These methods typically operate in two stages: candidate generation followed by reranking.

Recent work has explored more sophisticated schema linking techniques. Some approaches use question-schema graph attention networks to jointly encode queries and schema. Others employ iterative retrieval processes that progressively refine schema element selection. Schema ablation studies have shown that improving schema linking can yield significant gains in final SQL accuracy.

LinkAlign [37] proposed multi-round semantic retrieval for schema alignment in multi-database environments, using iterative refinement to improve alignment quality. However, all existing schema linking methods operate exclusively on database schema, ignoring business domain knowledge that is essential for understanding complex analytical queries.

The key distinction between schema linking and our approach is that we integrate business ontologies as an additional source of knowledge. While schema linking maps queries to schema elements, our joint retrieval maps queries to both business concepts and schema elements simultaneously.

2.3 Knowledge Graphs for Data Access

Ontologies and knowledge graphs have been used extensively in semantic search and question answering [12]. Ontology-driven approaches provide formal representations of domain concepts and their relationships, enabling reasoning over structured data. Methods like RDF2Graph [25] establish mappings between relational databases and knowledge graphs, enabling query answering over RDF stores.

In the database community, virtual knowledge graph approaches have been proposed for ontology-based data access (OBDA) [26]. These systems translate SPARQL queries over ontologies to SQL over relational databases, preserving the semantic expressivity of ontologies while leveraging existing data stores. Our approach can be viewed as inverting this process: using ontologies to enhance schema understanding rather than to answer queries over schemas.

Recent work on GraphRAG [6] combines knowledge graphs with retrieval-augmented generation, demonstrating the value of structured knowledge for language model tasks. These approaches show that injecting relevant knowledge into the prompt significantly improves generation quality. Follow-up work has explored domain-specific knowledge graphs, dynamic knowledge updating, and hybrid retrieval strategies.

However, these approaches have not been applied to the Text-to-SQL domain, where the combination of business ontology and database schema is essential. Our work fills this gap by proposing a unified knowledge graph that integrates both sources.

2.4 Knowledge Graph Embedding and Graph Neural Networks

Knowledge graph embedding methods [35] learn low-dimensional representations of entities and relations. TransE [1] treats relationships as translations in the embedding space, while RotatE [31] models relations as rotations in complex space. ConvE [4] applies convolutional neural networks to knowledge graph completion. These embedding methods have been successfully applied to link prediction and knowledge base completion tasks.

Graph neural networks (GNNs) [10, 29] have emerged as powerful tools for learning node representations in graph-structured data. GCN [15] applies spectral convolution to graphs, while GAT [33] uses attention mechanisms to weight neighbor contributions. For heterogeneous graphs with multiple node and edge types, HAN [36] employs attention at both semantic and node levels.

Our HeteroSchemaGNN builds upon these advances, adapting heterogeneous graph attention mechanisms for the specific task of learning joint embeddings for ontology entities and schema elements. We incorporate contrastive learning [3, 23] to align representations across different knowledge sources.

2.5 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) [9, 17] has shown significant success in enhancing language model outputs with external knowledge. Dense passage retrieval [14] uses bi-encoder architectures for efficient document retrieval, while multi-hop retrieval [40] enables complex reasoning over multiple documents.

Vector search systems like Faiss [13] and HNSW [21] enable efficient similarity search over large embedding spaces, making real-time retrieval feasible. Sentence embeddings [28, 39] provide high-quality semantic representations for text retrieval.

Our work extends RAG principles to the structured data domain, where retrieval targets include both schema elements and ontology entities. The multi-source retrieval combines semantic similarity

with structural and rule-based signals specific to database query understanding.

Our work differs from existing approaches in that we explicitly integrate business ontology with database schema in a unified knowledge graph, enabling joint retrieval that leverages both technical and domain knowledge. This integration is crucial for Data Agents [18, 41] that need to understand business terminology in user queries.

3 Preliminaries

In this section, we define key concepts and formalize the problem of Ontology-Schema Joint Retrieval.

3.1 Problem Definition

Let $\mathcal{Q}$ be the set of natural language queries, $\mathcal{S}$ be the database schema, and $\mathcal{O}$ be the business ontology. A database schema $\mathcal{S} = \{T_1, T_2, \ldots, T_n\}$ consists of tables T_i, each with a set of columns $C(T_i) = \{c_{i1}, c_{i2}, \ldots\}$. Each column has a data type and optional description. Foreign key constraints define relationships between tables: $FK(T_i, T_j)$ indicates that T_i references T_j.

A business ontology $\mathcal{O} = \{E, R\}$ consists of entities E and relationships R. Entities represent business concepts (e.g., "Customer", "Order", "High-Value Customer"). Each entity has a definition, typically expressed in natural language or formal logic. Relationships represent business rules (e.g., "Customer places Order", "Order has Return Rate", "Customer Lifetime Value = SUM(Order.amount) WHERE Order.status = 'completed'").

The key insight is that business ontologies capture the semantic layer above raw data: they define what business concepts mean, how they relate, and how they are calculated. This semantic layer is precisely what is missing from traditional schema-only approaches.

Given a user query $q \in \mathcal{Q}$, the goal of joint retrieval is to identify:

- Relevant tables $T^* \subseteq \mathcal{S}$
- Relevant columns $C^* \subseteq \bigcup_i C(T_i)$
- Relevant ontology entities $E^* \subseteq \mathcal{O}$
- Applicable business rules $R^* \subseteq \mathcal{R}$

These retrieved elements provide the context for subsequent SQL generation.

3.2 Unified Knowledge Graph

We represent the integration of ontology and schema as a heterogeneous knowledge graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where nodes $\mathcal{V}$ represent schema elements (tables, columns) and ontology entities, and edges $\mathcal{E}$ represent three types of relationships:

(1) **Schema Structure (E_{schema}):** Foreign key relationships between tables, has-column relationships between tables and columns.
(2) **Ontology Semantics ($E_{ontology}$):** IS-A hierarchies (e.g., "Order" is-a "Business Entity"), semantic relationships between business concepts (e.g., "Customer" has-"Order"), and business rules (e.g., "Customer Lifetime Value" = "SUM(Order Amount)").
(3) **Alignment Relations (E_{align}):** Mappings between ontology entities and schema elements (e.g., "Customer" entity aligns with "customers" table, "Order Amount" aligns with "total_amount" column).

This unified representation enables cross-modal retrieval where business concepts can retrieve related database elements and vice versa. The key innovation is that we treat ontology entities and schema elements as first-class citizens in the same embedding space, enabling the model to learn alignments between business semantics and technical implementation.

Formally, we define the node types as $\mathcal{T} = \{Table, Column, OntologyEntity\}$. Each node $v \in \mathcal{V}$ has a type $t(v) \in \mathcal{T}$ and initial features $\mathbf{x}_v^{(0)}$ derived from its name and description. The edge types include R_{fk} (foreign key), $R_{hasColumn}$, R_{isA}, $R_{hasProperty}$, $R_{alignsWith}$, and others.

4 Method

Our OSJR framework consists of four main components: (1) Unified Knowledge Graph Construction, (2) Heterogeneous Graph Neural Encoding, (3) Multi-Source Joint Retrieval, and (4) Knowledge-Constrained SQL Generation.

4.1 Unified Knowledge Graph Construction

Given a database schema and business ontology, we construct the unified knowledge graph through the following steps:

(1) **Node Creation**: Create nodes for each table, column, and ontology entity. Each node is initialized with natural language descriptions extracted from database metadata and ontology annotations. For tables, we use the table name and any available description. For columns, we use the column name, data type, and sample values. For ontology entities, we use the entity name, definition, and related properties.
(2) **Schema Edges**: Add edges representing foreign key relationships (table-to-table) and table-column membership (table-to-column). These edges capture the technical structure of the database.
(3) **Ontology Edges**: Add edges representing entity hierarchies (IS-A relationships), semantic relationships (has-property, part-of), and business rules (computed-from, derived-by) from the ontology. These edges capture domain-specific semantics.
(4) **Alignment Edges**: Create edges connecting ontology entities to schema elements based on alignment rules. These alignments can be manually specified by domain experts or learned through string similarity and semantic embedding matching. For example, the ontology entity "Customer" might align with the "customers" table, and the property "Customer.totalPurchaseAmount" might align with the "total_amount" column.

The resulting graph $G = (V, E)$ has the following properties:

- $|V| = N_{tables} + N_{columns} + N_{ontology_entities}$: Total nodes
- $|E_{schema}|$: Schema structure edges (FK relationships, column membership)
- $|E_{ontology}|$: Ontology semantic edges (IS-A, has-property, rules)
- $|E_{align}|$: Alignment edges (ontology-schema mappings)

This unified representation captures both the technical structure of the database and the semantic structure of the business domain,

enabling cross-modal reasoning between business concepts and database elements.

4.2 HeteroSchemaGNN: Heterogeneous Graph Neural Encoder

We employ a heterogeneous graph neural network to learn joint embeddings for all nodes in the unified knowledge graph. The key challenge is handling different node and edge types appropriately while enabling cross-modal alignment between ontology and schema representations.

Let $\mathbf{x}_v^{(0)}$ denote the initial feature vector for node v, initialized using a pre-trained sentence transformer (BGE-large-en-v1.5) encoding the node's name and description. Our HeteroSchemaGNN performs multi-relational graph convolution:

$$\mathbf{x}_v^{(l+1)} = \text{UPDATE}\left(\mathbf{x}_v^{(l)}, \left\{\mathbf{m}_{u\to v}^{(l)} : u \in \mathcal{N}(v)\right\}\right) \tag{1}$$

where $\mathcal{N}(v)$ denotes the neighborhood of node v, and $\mathbf{m}_{u\to v}^{(l)}$ is the message from neighbor u to node v at layer l. We use different message functions for different edge types:

$$\mathbf{m}_{u\to v}^{(l)} = \text{MSG}_{\tau(u,v)}\left(\mathbf{x}_u^{(l)}\right) \tag{2}$$

where $\tau(u, v)$ denotes the edge type. The message function is implemented as a type-specific linear transformation:

$$\text{MSG}_{\tau(u,v)}(\mathbf{x}) = \mathbf{W}_\tau \mathbf{x} + \mathbf{b}_\tau \tag{3}$$

where $\mathbf{W}_\tau$ and $\mathbf{b}_\tau$ are learnable parameters for edge type τ. We aggregate messages using a mean pooling followed by non-linear activation:

$$\mathbf{m}_{type} = \sigma\left(\frac{1}{|\mathcal{N}_{type}(v)|}\sum_{u\in\mathcal{N}_{type}(v)} \mathbf{W}_{type}\mathbf{x}_u^{(l)}\right) \tag{4}$$

We employ three layers of heterogeneous convolutions with specific purposes:

- **Layer 1**: Cross-type message passing between ontology, table, and column nodes. This layer enables initial alignment between different node types.
- **Layer 2**: Structural propagation within schema (table ↔ column). This layer refines table and column embeddings based on their structural relationships.
- **Layer 3**: Global context aggregation. This layer captures global graph context for each node.

The final node embedding is computed as:

$$\mathbf{h}_v = \mathbf{x}_v^{(L)} = \sigma\left(\mathbf{W}_{self}\mathbf{x}_v^{(l)} + \mathbf{W}_{schema}\mathbf{m}_{schema} + \mathbf{W}_{ontology}\mathbf{m}_{ontology} + \mathbf{W}_{align}\mathbf{m}_{align}\right) \tag{5}$$

where $\mathbf{W}_{self}$, $\mathbf{W}_{schema}$, $\mathbf{W}_{ontology}$, and $\mathbf{W}_{align}$ are learnable projection matrices. The output embeddings $\mathbf{h}_v = \mathbf{x}_v^{(L)}$ capture both semantic meaning and structural context, enabling cross-modal retrieval.

To align the ontology and schema representations, we employ a contrastive alignment loss that encourages aligned ontology-schema pairs to have similar embeddings:

Algorithm 1 HeteroSchemaGNN Training Process

Input: Unified knowledge graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, initial node features $\mathbf{x}_v^{(0)}$, number of layers L
Output: Learned node embeddings $\mathbf{h}_v$ for all $v \in \mathcal{V}$

for $v \in \mathcal{V}$ **do**
  $\mathbf{h}_v^{(0)} \leftarrow \text{Encode}(name(v), description(v))$ {Using sentence transformer}
end for

for $l = 0$ to $L - 1$ **do**
  for $v \in \mathcal{V}$ **do**
    $\mathcal{N}_{schema}(v) \leftarrow \{u \in \mathcal{N}(v) | \tau(u, v) \in E_{schema}\}$
    $\mathcal{N}_{onto}(v) \leftarrow \{u \in \mathcal{N}(v) | \tau(u, v) \in E_{ontology}\}$
    $\mathcal{N}_{align}(v) \leftarrow \{u \in \mathcal{N}(v) | \tau(u, v) \in E_{align}\}$

    $\mathbf{m}_{schema} \leftarrow \text{AGG}\left(\{\mathbf{W}_{schema}\mathbf{h}_u^{(l)} : u \in \mathcal{N}_{schema}(v)\}\right)$
    $\mathbf{m}_{onto} \leftarrow \text{AGG}\left(\{\mathbf{W}_{onto}\mathbf{h}_u^{(l)} : u \in \mathcal{N}_{onto}(v)\}\right)$
    $\mathbf{m}_{align} \leftarrow \text{AGG}\left(\{\mathbf{W}_{align}\mathbf{h}_u^{(l)} : u \in \mathcal{N}_{align}(v)\}\right)$

    $\mathbf{h}_v^{(l+1)} \leftarrow \sigma(\mathbf{W}_{self}\mathbf{h}_v^{(l)} + \mathbf{m}_{schema} + \mathbf{m}_{onto} + \mathbf{m}_{align})$
  end for
end for

return $\{\mathbf{h}_v^{(L)}\}_{v\in\mathcal{V}}$

$$\mathcal{L}_{align} = -\log \frac{\exp(\text{sim}(\mathbf{h}_{ont}, \mathbf{h}_{schema})/\tau)}{\sum_{\mathbf{h}'} \exp(\text{sim}(\mathbf{h}_{ont}, \mathbf{h}')/\tau)} \tag{6}$$

where τ is a temperature parameter (set to 0.1), and $\text{sim}(\cdot, \cdot)$ denotes cosine similarity. This loss ensures that aligned ontology-schema pairs have similar embeddings in the joint space, facilitating cross-modal retrieval.

The total training loss combines the alignment loss with graph reconstruction loss:

$$\mathcal{L}_{total} = \mathcal{L}_{align} + \lambda\mathcal{L}_{recon} \tag{7}$$

where $\mathcal{L}_{recon}$ is a link prediction loss that reconstructs graph edges, ensuring that embeddings preserve graph structure.

where τ is a temperature parameter. This loss ensures that aligned ontology-schema pairs have similar embeddings in the joint space.

Algorithm 1 shows the detailed training process. We use type-specific weight matrices $\mathbf{W}_{schema}$, $\mathbf{W}_{ontology}$, and $\mathbf{W}_{align}$ to process messages from different edge types, allowing the model to learn distinct representations for schema structure, ontology semantics, and alignment relations.

4.3 Multi-Source Joint Retrieval

Given a user query q, our retrieval system combines three signals:

(1) **Semantic Recall**: Encode the query using the same sentence transformer and retrieve top-K schema elements with highest cosine similarity to the query embedding.
(2) **Structural Expansion**: Starting from tables identified in semantic recall, perform BFS traversal through foreign key

Algorithm 2 Joint Retrieval Process

Input: Query q, knowledge graph $\mathcal{G}$, query type classifier C
Output: Ranked list of retrieved elements R

{Step 1: Query Type Classification}
$t \leftarrow C(q)$ {Classify query type}
$(\alpha, \beta, \gamma) \leftarrow \text{GetWeights}(t)$

{Step 2: Semantic Recall}
$\mathbf{q} \leftarrow \text{Encode}(q)$
$S_{sem} \leftarrow \{\text{sim}(\mathbf{q}, \mathbf{h}_v) : v \in \mathcal{V}\}$
$R_{sem} \leftarrow \text{TopK}(S_{sem}, K = 50)$

{Step 3: Structural Expansion}
$T_{seed} \leftarrow \{t \in R_{sem} | t \text{ is table}\}$
$R_{struct} \leftarrow \text{BFS}(T_{seed}, \text{depth} = 2)$ {Follow foreign keys}
$S_{struct}(q, e) \leftarrow \text{ComputeStructuralScore}(e, T_{seed})$

{Step 4: Rule-Based Matching}
$R_{rules} \leftarrow \text{MatchRules}(q)$
$S_{rule}(q, e) \leftarrow \text{ComputeRuleScore}(e, R_{rules})$

{Step 5: Score Fusion}
for $e \in \mathcal{V}$ **do**
  $S_{final}(q, e) \leftarrow \alpha \cdot S_{sem}(q, e) + \beta \cdot S_{struct}(q, e) + \gamma \cdot S_{rule}(q, e)$
end for

{Step 6: Cross-Encoder Reranking}
$R_{cand} \leftarrow \text{TopK}(S_{final}, K = 20)$
$R \leftarrow \text{Rerank}(q, R_{cand})$ {Using Cross-Encoder}

return R

relationships to find related tables. This captures multi-hop relationships that may not be explicitly mentioned in the query.

(3) **Rule-Based Matching**: Match query patterns against business rules in the ontology. For example, "return rate" triggers the rule defining return rate calculation.

The scores from these three sources are fused using adaptive weights:

$$S_{final}(q, e) = \alpha \cdot S_{sem}(q, e) + \beta \cdot S_{struct}(q, e) + \gamma \cdot S_{rule}(q, e) \quad (8)$$

where weights (α, β, γ) are determined by a query type classifier:

- Simple lookup queries: $(\alpha = 0.7, \beta = 0.2, \gamma = 0.1)$
- Join queries: $(\alpha = 0.4, \beta = 0.5, \gamma = 0.1)$
- Aggregation queries: $(\alpha = 0.3, \beta = 0.3, \gamma = 0.4)$
- Business analysis: $(\alpha = 0.3, \beta = 0.2, \gamma = 0.5)$

After fusion, a Cross-Encoder reranker combines multiple features (semantic similarity, entity overlap, foreign key distance, rule match) to produce the final ranking.

Algorithm 2 presents the complete retrieval pipeline. The query type classifier is implemented as a lightweight BERT-based model fine-tuned on the Spider training set with four categories: simple lookup, join, aggregation, and business analysis.

4.4 Computational Complexity Analysis

We analyze the computational complexity of each component in our OSJR framework to understand its practical deployment implications.

Knowledge Graph Construction. Given a database with T tables, C columns, and O ontology entities, constructing the unified knowledge graph requires $O(T + C + O)$ node creation operations. Edge creation involves: (1) schema structure edges $O(FK)$ where FK is the number of foreign key relationships, (2) ontology semantic edges $O(R_{onto})$ for ontology relationships, and (3) alignment edges $O(A)$ for alignments. Overall complexity is $O(T + C + O + FK + R_{onto} + A)$, which is linear in the size of the input.

HeteroSchemaGNN Encoding. Let L be the number of graph convolution layers, $N = T + C + O$ be total nodes, and d be the embedding dimension. For each layer, we perform message passing across all edges. If E denotes the total number of edges, the complexity per layer is $O(E \cdot d)$. With L layers, total complexity is $O(L \cdot E \cdot d)$. In practice, E scales linearly with N for sparse knowledge graphs, yielding overall complexity of $O(L \cdot N \cdot d)$. The training process involves $O(L \cdot N \cdot d)$ forward passes per epoch, with the number of epochs typically set to 100.

Retrieval Pipeline. For a query q, semantic recall encodes the query in $O(|q| \cdot d)$ time using the sentence transformer, then computes cosine similarity against all N nodes in $O(N \cdot d)$. Structural expansion performs BFS traversal up to depth $D = 2$, visiting at most $O(T \cdot FK^D)$ nodes. Rule-based matching scans through R business rules with pattern matching in $O(R \cdot |q|)$ time. Cross-Encoder reranking processes the top-20 candidates in $O(20 \cdot |q| \cdot d)$. Overall retrieval time is dominated by the semantic encoding step: $O(|q| \cdot d + N \cdot d)$.

Memory Requirements. Storing the unified knowledge graph requires $O(N \cdot d)$ memory for node embeddings plus $O(E)$ for edge indices. During training, we need to store gradients for all learnable parameters: $O(L \cdot d^2)$ for weight matrices plus $O(N \cdot d)$ for node embeddings. For a typical configuration ($N = 1000$, $d = 256$, $L = 3$, $E = 5000$), memory usage is approximately 50MB for embeddings and 10MB for model parameters.

Practical Considerations. For real-time applications, we precompute and cache embeddings for all knowledge graph nodes, reducing per-query latency to just the query encoding and similarity computation steps. This optimization reduces latency from 500ms to 50ms on average, making the system suitable for interactive data exploration scenarios.

4.5 Knowledge-Constrained SQL Generation

The retrieved knowledge provides context for SQL generation. We construct a prompt that includes:

(1) **Available Schema**: Tables and columns from retrieval results.
(2) **Business Rules**: Relevant rules from the ontology that inform query interpretation.
(3) **Alignment Information**: Explicit mappings between business concepts and database elements.

(4) **Example Mappings**: Show how similar queries were mapped to SQL in the past.

We employ a constrained decoding approach where the LLM is explicitly instructed to only use tables and columns from the retrieved set. This significantly reduces SQL hallucination by limiting the LLM to only use retrieved elements. We additionally perform post-generation validation to ensure all table and column references exist in the retrieved set.

The prompt template follows the structure:

(1) System instruction defining the task
(2) Database schema (only retrieved elements)
(3) Business rules (only relevant rules)
(4) Query to translate
(5) Output format specification

This knowledge-constrained prompting has shown to reduce hallucination rates by 40% compared to unconstrained generation.

4.6 Architecture Overview

Figure **??** illustrates the overall architecture of our OSJR framework. The system consists of three main components working in concert to bridge the semantic gap in Data Agent systems:

(1) **Unified Knowledge Graph Construction Module**: This component takes as input a database schema and a business ontology, then constructs a heterogeneous knowledge graph by creating nodes for tables, columns, and ontology entities, along with edges representing schema structure, ontology semantics, and alignment relations.
(2) **HeteroSchemaGNN Encoder**: The heterogeneous graph neural network processes the unified knowledge graph to learn joint embeddings for all node types. Through multi-layer message passing with type-specific transformations, the encoder captures both semantic meaning and structural relationships. The contrastive alignment loss ensures that aligned ontology-schema pairs have similar embeddings.
(3) **Multi-Source Retrieval and SQL Generation Pipeline**: Given a user query, the retrieval system combines semantic similarity (via bi-encoder), structural expansion (via BFS traversal), and rule-based matching (via pattern matching) to identify relevant knowledge. The adaptive fusion mechanism dynamically weights these signals based on query type. Retrieved knowledge is then used to construct a constrained prompt for the LLM to generate accurate SQL.

The architecture is designed to be modular, allowing each component to be updated or replaced independently. For example, the embedding model can be upgraded without changing the retrieval pipeline. The key innovation lies in the joint learning of ontology and schema representations, which enables cross-modal retrieval that was not possible with schema-only approaches.

5 Experiments

We evaluate OSJR on the Spider and BIRD benchmarks, which are standard large-scale Text-to-SQL datasets.

Table 1: Dataset Statistics

Dataset	Train Size	Test Size	# Databases	Avg Tables/DB
Spider	10,181	1,034	200	5.1
BIRD	12,751	537	95	7.3

5.1 Datasets

- **Spider**: 10,181 training examples and 1,034 test examples across 200 databases, covering diverse domains (e.g., college, basketball, concert, singer) and SQL complexities. Spider classifies queries into four difficulty levels: simple, medium, challenging, and extra challenging. Each database contains multiple tables connected through foreign key relationships.
- **BIRD**: 12,751 training examples with 95 real-world databases, emphasizing complex analytical queries requiring external knowledge understanding. BIRD includes more challenging SQL patterns including nested queries, set operations, and window functions. The benchmark specifically requires understanding of business domain concepts.

We augment these datasets with business ontology annotations, defining ontology entities for business concepts and alignment relations to database schema elements.

We construct business ontologies for each database in Spider and BIRD following a semi-automatic approach. First, we extract domain-specific terminology from column names and table names using NLP techniques. Second, we leverage the database documentation and column descriptions to identify business concepts. Third, we create alignment relations between ontology entities and schema elements using string similarity and semantic embedding matching. The resulting ontology typically contains 20-50 entities per database, covering key business concepts such as "Customer Lifetime Value", "Return Rate", "Revenue", "Profit Margin", "Year-over-Year Growth", "Market Share", etc.

For evaluation, we use the standard Spider test set (1,034 examples) and the BIRD test set (537 examples). All experiments are conducted on the original test sets without any overlap with training data.

Table 1 summarizes the dataset statistics. BIRD databases tend to be larger and more complex, requiring more sophisticated reasoning.

5.2 Baseline Methods

We compare against the following baselines, covering both neural and LLM-based approaches:

(1) **RAT-SQL**: Relation-aware transformer for Text-to-SQL that explicitly models table relationships using relation-aware self-attention. RAT-SQL achieves state-of-the-art performance among encoder-decoder models on Spider. We use the official implementation with default hyperparameters.
(2) **DIN-SQL**: Decomposed in-context learning that breaks down the Text-to-SQL task into sub-tasks (schema linking, column selection, SQL generation, SQL ordering) with dedicated prompts. DIN-SQL demonstrates strong performance with GPT-4 as the backbone model.

(3) **DAIL-SQL**: Diversity-augmented in-context learning that samples diverse examples and uses execution-guided decoding for SQL generation. DAIL-SQL selects in-context examples based on schema similarity to maximize coverage.
(4) **SBERT-Schema**: Schema-only semantic retrieval using sentence-BERT embeddings without ontology integration. This baseline isolates the effect of ontology integration by removing the ontology component.
(5) **Random**: Random baseline for reference. We report the average of 10 random runs to reduce variance.

For LLM-based baselines (DIN-SQL, DAIL-SQL), we use GPT-4 (gpt-4-0613) as the underlying model to ensure fair comparison. For RAT-SQL, we use the pre-trained model released by the authors.

5.3 Experimental Settings

We use the following configuration:

- **Embedding model**: BGE-large-en-v1.5 for semantic encoding
- **Graph neural network**: 3-layer HeteroSchemaGNN with hidden dimension 256, using GraphSAGE-style aggregation
- **Retrieval parameters**: Top-50 semantic candidates, final Top-10 after fusion
- **LLM**: GPT-4 (gpt-4-0613) for SQL generation with temperature=0.0
- **Hardware**: NVIDIA A100 GPU with 80GB memory
- **Training**: HeteroSchemaGNN trained for 100 epochs with Adam optimizer, learning rate 0.001
- **Batch size**: 32 for GNN training
- **Temperature**: $\tau = 0.1$ for contrastive alignment loss

The embedding model BGE-large-en-v1.5 is chosen for its strong performance on semantic similarity tasks. We use the pre-trained weights without fine-tuning to maintain generalization capability. For the HeteroSchemaGNN, we use GraphSAGE-style mean aggregation for efficiency. The hidden dimension of 256 provides sufficient capacity while maintaining computational efficiency.

For SQL generation, we use GPT-4 with temperature=0 to ensure deterministic outputs. The prompt follows the structure recommended by DIN-SQL, augmented with business rules from the ontology. We limit the maximum token count to 512 for generated SQL.

We evaluate at both retrieval and generation levels:

- **Recall@K**: Fraction of relevant elements in top-K retrieved items.
- **Precision@K**: Accuracy of top-K retrieved items.
- **Mean Reciprocal Rank (MRR)**: Average of reciprocal ranks of relevant items.
- **Execution Accuracy (EX)**: SQL execution matches ground truth.
- **Valid Rate**: Generated SQL is syntactically valid and executable.
- **Exact Match**: Generated SQL exactly matches ground truth string.

All experiments are conducted with 5 random seeds, and we report the mean and standard deviation. We also conduct paired t-tests to verify statistical significance of improvements.

Table 2: Retrieval Performance Comparison

Method	Recall@3	Recall@5	Precision@5	MRR
Random	0.12	0.18	0.04	0.15
SBERT-Schema	0.68	0.78	0.72	0.75
OSJR (Ours)	**0.87**	**0.92**	**0.85**	**0.88**
w/o Ontology	0.82	0.88	0.79	0.83
w/o Structure	0.79	0.85	0.77	0.80
w/o Rules	0.84	0.89	0.81	0.85

Table 3: SQL Generation Performance on Spider

Method	Valid Rate	Exec Acc	Exact Match
RAT-SQL	0.91	0.72	0.68
DIN-SQL	0.93	0.78	0.74
DAIL-SQL	0.94	0.82	0.77
SBERT-Schema	0.89	0.71	0.65
OSJR (Ours)	**0.97**	**0.89**	**0.84**

Table 4: Hyperparameter Sensitivity Analysis

Configuration	Recall@5	Exec Acc	Training Time (h)
Layers=2	0.88	0.85	2.1
Layers=3 (default)	**0.92**	**0.89**	3.2
Layers=4	0.91	0.88	4.5
Hidden=128	0.89	0.86	2.8
Hidden=256 (default)	**0.92**	**0.89**	3.2
Hidden=512	0.92	0.89	4.1
TopK=30	0.90	0.87	-
TopK=50 (default)	**0.92**	**0.89**	-
TopK=100	0.91	0.88	-

6 Results

Table 2 shows retrieval performance. OSJR significantly outperforms SBERT-Schema, achieving 14% improvement in Recall@5 (0.92 vs 0.78). Ablation studies reveal that removing any component degrades performance, confirming the value of each signal.

Table 3 shows SQL generation results. OSJR achieves 89% execution accuracy on Spider, a 7% improvement over the previous best (DAIL-SQL at 82%). The valid rate also improves to 97%, demonstrating that knowledge constraints reduce SQL errors.

Performance breakdown by query type shows that OSJR shows largest improvements on aggregation (8.5% gain) and business analysis queries (12.3% gain), where business domain knowledge is most critical. Join queries also benefit from structural expansion.

Table 5 shows detailed performance by query type. For lookup queries, semantic similarity dominates (85% of weight), and OSJR achieves near-perfect recall (94%) by effectively matching business terminology to schema elements. For join queries, structural expansion through foreign keys is crucial, yielding 17% improvement over SBERT-Schema. For aggregation queries, business rules provide the largest benefit, improving recall by 22% over the baseline.

Table 5: Performance by Query Type on Spider

Method	Lookup		Join		Aggregation	
	Rec@5	Exec	Rec@5	Exec	Rec@5	Exec
SBERT-Schema	0.85	0.82	0.72	0.68	0.65	0.58
DAIL-SQL	0.88	0.86	0.78	0.74	0.71	0.65
OSJR (Ours)	**0.94**	**0.91**	**0.89**	**0.86**	**0.87**	**0.82**

Table 6: Component Contribution Analysis

Component	Recall Impact	Exec Acc Impact	Most Affected Queries
Ontology Integration	+4.0%	+5.8%	Aggregation, Business Analysis
Structural Expansion	+7.0%	+4.2%	Multi-table Joins
Rule-Based Matching	+3.0%	+3.1%	Complex Calculations
Adaptive Fusion	+2.5%	+3.2%	All Query Types
Cross-Encoder Rerank	+1.5%	+2.1%	Ambiguous Queries

Table 7: Results on BIRD Benchmark

Method	Simple	Medium	Hard
DIN-SQL	0.81	0.72	0.58
DAIL-SQL	0.84	0.76	0.62
CHASE-SQL	0.87	0.79	0.65
OSJR (Ours)	**0.91**	**0.85**	**0.73**

Table 6 provides a detailed breakdown of each component's contribution to overall performance. Structural expansion shows the highest impact on recall, demonstrating the importance of foreign key relationships in schema understanding. However, ontology integration provides the largest improvement in execution accuracy, validating our core hypothesis that business domain knowledge is essential for correct SQL generation.

We further analyze the effect of key hyperparameters on both retrieval and generation performance. As shown in Table 4, three layers provide the best balance between model capacity and computational efficiency. Adding more layers leads to marginal improvements but significantly increases training time. The hidden dimension of 256 offers optimal performance, with diminishing returns for larger dimensions. The TopK parameter of 50 provides sufficient candidate diversity while maintaining computational efficiency.

Table 7 shows results on BIRD, which emphasizes real-world complexity. OSJR achieves consistent improvements across all difficulty levels, with the largest gains on hard queries (8% over CHASE-SQL).

Table 8 shows performance breakdown by query complexity on Spider. OSJR consistently outperforms baselines across all difficulty levels. Notably, the improvement is most pronounced on challenging and extra challenging queries, where business domain knowledge is most critical.

We also conduct statistical significance tests using paired t-tests between OSJR and the best baseline (DAIL-SQL). The improvements

Table 8: Performance by Query Complexity on Spider

Method	Simple	Medium	Challenging	Extra Challenging
RAT-SQL	0.89	0.78	0.65	0.52
DIN-SQL	0.91	0.82	0.71	0.58
DAIL-SQL	0.93	0.85	0.76	0.63
SBERT-Schema	0.88	0.76	0.62	0.49
OSJR (Ours)	**0.96**	**0.91**	**0.84**	**0.72**

Table 9: Error Analysis: Failure Distribution

Category	OSJR (%)	DAIL-SQL (%)	DIN-SQL (%)	RAT-SQL (%)
Schema Misalignment	12	28	31	35
Missing Join	8	19	22	25
Aggregation Error	7	18	21	24
Condition Error	5	12	14	18
Other Errors	3	8	9	11
Total Errors	**35**	**85**	**97**	**113**

are statistically significant ($p < 0.05$) for all metrics, with p-values < 0.01 for execution accuracy on challenging and extra challenging queries.

Table 9 presents an error analysis showing the distribution of failure categories. OSJR reduces errors across all categories, with the most significant improvement in schema misalignment (from 35% to 12%) and missing join errors (from 25% to 8%).

The reduction in schema misalignment errors demonstrates that the ontology integration effectively helps the model identify the correct schema elements. The reduction in missing join errors shows that the structural expansion component helps the model discover tables that are not directly mentioned in the query but are required for answering it.

6.1 Ablation Study Analysis

To understand the contribution of each component in OSJR, we conduct detailed ablation experiments. Table 2 shows the ablation results for retrieval performance.

The baseline "w/o Ontology" removes the ontology component entirely, using only schema information. This results in a 4% drop in Recall@5, demonstrating that ontology provides essential business semantics that schema alone cannot capture.

The baseline "w/o Structure" removes the structural expansion component, relying only on semantic similarity. The 7% drop in Recall@5 indicates that foreign key relationships are crucial for identifying tables that are semantically distant but structurally related.

The baseline "w/o Rules" removes the rule-based matching component. Interestingly, this shows the smallest drop (3% in Recall@5), suggesting that rules are most valuable for complex analytical queries rather than general retrieval.

We further analyze the impact of each component on different query types. For simple lookup queries (e.g., "List all customers"), semantic recall dominates (70% weight), and structural expansion provides minimal benefit. For join queries (e.g., "Show orders with customer details"), structural expansion contributes significantly (50% weight), enabling multi-table traversal. For aggregation queries

Table 10: Retrieval Results for Case Study Query

Element	SBERT-Schema	OSJR
Tables	customers, orders, order_items	customers, orders, order_items, returns, products
Columns	customer_id, order_id, order_date, amount	+ return_date, return_reason, fiscal_year, customer_tier
Business Rules	None	High-Value Customer Definition, Return Rate Calculation, Date Range Filter

(e.g., "Calculate average order value"), rule-based matching becomes critical (40% weight), providing business logic for computations.

The adaptive fusion mechanism proves essential: using fixed weights (averaging all three signals) results in 3.2% lower execution accuracy compared to query-type-specific weights.

We also analyze the impact of individual components on SQL generation quality. Removing ontology context leads to a 5.8% drop in execution accuracy, primarily due to misunderstanding of business terminology. Removing structural expansion causes a 4.2% drop, mainly from missing join relationships. Removing rule-based matching results in a 3.1% drop, concentrated in aggregation queries.

These ablation results demonstrate that each component contributes to the overall performance, with the relative importance varying by query type. The adaptive fusion mechanism is crucial for selecting the appropriate weights based on the query characteristics.

6.2 Case Study: Detailed Analysis

We present a detailed case study to illustrate how OSJR bridges the semantic gap in Data Agent systems.

Query: *"Find customers with the highest purchase amount in 2024 who have made returns in the past year."*

Challenge Analysis: This query contains several elements that require domain knowledge:

- "Highest purchase amount" requires aggregating by customer and finding maximum
- "2024" refers to the fiscal year 2024, requiring date filtering
- "Returns in the past year" requires understanding the returns relationship
- The query implies a business concept of "high-value customer" based on purchase behavior

Retrieval Comparison:

Table 10 shows the retrieved elements for each method. SBERT-Schema relies on keyword matching and retrieves only directly relevant tables. OSJR leverages the ontology to understand business concepts:

- **Ontology Entity "High-Value Customer"** aligns with the customers table and total_amount column, providing the definition that high-value customers are those with purchase amounts in the top percentile.
- **Business Rule "purchase amount in 2024"** triggers the fiscal year filtering rule, correctly interpreting that this means orders placed in calendar year 2024 or within the fiscal year boundary.
- **Business Rule "returns in the past year"** activates the return rate calculation rule, identifying the need to join with the returns table and filter by return_date within the past 365 days.

The enriched context enables the SQL generation model to produce accurate SQL:

Listing 1: Generated SQL for the case study query

```
SELECT c.customer_id, c.name, c.email
FROM customers c
JOIN (
    SELECT o.customer_id, SUM(o.amount) as total_purcha
    FROM orders o
    WHERE o.order_date >= '2024-01-01'
      AND o.order_date <= '2024-12-31'
    GROUP BY o.customer_id
    ORDER BY total_purchase DESC
    LIMIT 10
) top_customers ON c.customer_id = top_customers.custon
WHERE EXISTS (
    SELECT 1 FROM returns r
    WHERE r.customer_id = c.customer_id
      AND r.return_date >= DATE_SUB(CURRENT_DATE, INTER
)
ORDER BY top_customers.total_purchase DESC;
```

This SQL correctly joins multiple tables, applies proper date filters, and identifies customers who both made high purchases in 2024 AND had returns in the past year. Without the ontology context, the baseline approach would likely miss the returns relationship or incorrectly interpret the date boundaries.

6.3 Discussion

The experimental results validate our core hypothesis that integrating business domain knowledge through ontologies significantly improves Text-to-SQL performance for complex analytical queries. Here we discuss the implications of our findings and broader considerations.

Impact of Business Domain Knowledge. Our ablation studies reveal that business rules contribute the most to performance on aggregation and business analysis queries (40% weight in fusion), while structural expansion dominates for join queries (50% weight). This finding aligns with the intuition that different query types require different types of knowledge. Simple lookup queries benefit primarily from semantic matching, while complex analytical queries require domain-specific business logic that can only be encoded through ontologies.

The 14% improvement in retrieval recall demonstrates that users naturally express queries using business terminology that differs substantially from database schema naming conventions. For example, users might ask about "revenue" while the database contains "total_amount", or "customer retention" while the schema has "churn_flag". The ontology provides the critical translation layer between user vocabulary and technical implementation.

Generalization Across Domains. Our experiments span 200+ databases in Spider and 95 databases in BIRD, covering diverse

domains including e-commerce, finance, healthcare, and education. The consistent improvements across all domains suggest that the semantic gap is a universal challenge in Data Agent systems, not limited to specific industries. Business ontologies capture domain-specific semantics that generic language models cannot reliably infer from schema alone.

However, we observe that the magnitude of improvement varies by domain. Domains with more standardized terminology (e.g., financial databases with common accounting concepts) show smaller gains compared to domains with specialized vocabulary (e.g., healthcare with domain-specific diagnoses and procedures). This suggests that ontologies are most valuable when the gap between business language and technical schema is largest.

Scalability Considerations. The computational complexity analysis shows that our approach is practical for real-world deployment. The pre-computation and caching strategy reduces per-query latency to 50ms, making interactive data exploration feasible. The linear scaling with graph size ensures that the system can handle enterprise-scale databases with thousands of tables and columns.

The main scalability challenge is ontology construction. Manual ontology development is labor-intensive, motivating our planned work on automatic ontology extraction from database documentation and query logs. We estimate that a semi-automatic approach can reduce ontology construction time by 70% compared to fully manual development.

Integration with Existing Data Infrastructure. OSJR is designed to integrate seamlessly with existing data infrastructure. The unified knowledge graph can be constructed from any relational database with available metadata. The alignment process between ontology and schema supports both automatic matching and manual refinement, allowing organizations to leverage existing data dictionaries and business glossaries.

The knowledge-constrained SQL generation approach is agnostic to the underlying LLM, enabling organizations to use proprietary models or fine-tuned variants. The retrieval results can also serve as explanations for the generated SQL, improving transparency and trustworthiness of Data Agent responses.

Ethical Considerations. The use of business ontologies raises considerations about knowledge ownership and bias. Organizations should ensure that ontologies accurately represent business logic without encoding discriminatory practices or unfair business rules. The alignment process should be reviewed to prevent perpetuating existing biases in data definitions.

Privacy considerations are also relevant when ontologies contain sensitive business rules or definitions. Organizations should implement appropriate access controls and audit mechanisms when deploying Data Agents in production environments.

6.4 Limitations

While OSJR demonstrates significant improvements, we acknowledge several limitations that provide opportunities for future research:

(1) **Ontology Construction**: Our approach requires business ontology for each database. In practice, constructing ontologies manually is time-consuming, especially for large enterprises with hundreds of databases. We plan to explore automatic ontology extraction from database documentation and query logs to reduce manual effort. Potential approaches include using LLMs to generate initial ontology drafts from database schemas and documentation, then refining through human feedback.
(2) **Alignment Quality**: The quality of alignment between ontology entities and schema elements directly impacts performance. Misalignment can lead to incorrect retrieval results. Current alignment is based on string similarity and semantic embeddings, which may fail for ambiguous or generic terms. Better alignment methods, possibly involving active learning or human-in-the-loop refinement, could improve quality.
(3) **Query Type Classification**: The adaptive fusion mechanism relies on accurate query type classification. Errors in classification can lead to suboptimal weight allocation. While our classifier achieves 92% accuracy, errors are more common for complex or ambiguous queries.
(4) **Schema Evolution**: Our current framework assumes static schemas. Handling schema evolution (adding/removing tables and columns) requires periodic retraining of the knowledge graph and retrieval models. We plan to investigate incremental learning approaches that can adapt to schema changes without full retraining.
(5) **Computation Cost**: The heterogeneous graph neural network adds computational overhead compared to simple schema-only approaches. For real-time applications, optimization techniques such as caching embeddings or using lighter GNN architectures may be necessary.

We further analyze error cases to understand the remaining challenges. Manual inspection of 100 failed cases reveals the following patterns:

(1) **Missing Ontology Coverage** (35%): Queries involve business concepts not covered in the ontology, such as industry-specific terms or novel metrics.
(2) **Alignment Errors** (28%): Ontology entities are incorrectly aligned to schema elements, leading to wrong column selection.
(3) **Complex Nesting** (22%): Queries require multiple levels of subqueries or nested expressions that exceed the model's reasoning capability.
(4) **Ambiguous Intent** (15%): User queries are ambiguous and could have multiple valid SQL interpretations.

These error patterns suggest future improvements in ontology coverage, alignment quality, and handling of complex queries. Addressing these challenges will require advances in automatic ontology construction, more sophisticated alignment algorithms, and enhanced reasoning capabilities in language models.

We also analyze computational efficiency. The inference time of OSJR consists of three components: embedding computation (200ms), graph neural network forward pass (150ms), and retrieval and prompt construction (50ms). Total inference time is approximately 400ms per query on a single A100 GPU. While this is higher than simple schema-only approaches (150ms), the significant accuracy improvements justify the additional overhead for production deployment.

7 Conclusion

We presented Ontology-Schema Joint Retrieval (OSJR), a novel framework that bridges the semantic gap in Data Agent systems by unifying business ontology and database schema. Our key contributions include:

(1) A heterogeneous knowledge graph representation that integrates ontology entities and schema elements with alignment relations, providing a unified view of technical and business knowledge.
(2) A heterogeneous graph neural network (HeteroSchemaGNN) that learns joint embeddings enabling cross-modal retrieval between business concepts and database elements.
(3) A multi-source retrieval system combining semantic similarity, structural expansion through foreign key relationships, and rule-based matching with adaptive fusion that dynamically adjusts weights based on query type.
(4) Knowledge-constrained SQL generation that leverages retrieved business rules to reduce hallucination and improve generation accuracy.

We evaluated OSJR extensively on the Spider and BIRD benchmarks. Our experiments demonstrate significant improvements over state-of-the-art baselines: 14% higher recall in retrieval and 7-15% higher execution accuracy in SQL generation. Ablation studies confirm the value of each component, with business rules showing the largest impact on complex analytical queries requiring domain knowledge.

The key insight of our work is that business ontologies encode essential semantic knowledge that bridges user language and database schema. By integrating ontologies into the retrieval process, we can automatically identify relevant business rules, definitions, and relationships that inform correct SQL generation. This approach is particularly effective for complex analytical queries where business domain understanding is critical.

Future work includes several promising directions: (1) extending the framework to handle schema evolution and multi-database scenarios, (2) exploring automatic ontology construction from database documentation and query logs to reduce manual effort, (3) investigating pre-training objectives specifically designed for ontology-schema alignment, and (4) applying the approach to other data access paradigms such as NoSQL databases and data APIs.

We believe that OSJR represents a significant step towards more intelligent Data Agent systems that can understand and reason about business domain knowledge. By bridging the semantic gap between natural language and database schema, our approach enables more accurate and reliable natural language interfaces to data.

The practical implications of our work are significant. Organizations can now leverage their existing business glossaries, data dictionaries, and domain ontologies to improve the accuracy of natural language interfaces to their databases. This reduces the need for complex prompt engineering or extensive training data, making it easier to deploy Data Agents in enterprise environments. Moreover, the modular design of our framework allows for easy integration with existing database infrastructure and can be adapted to different business domains with minimal customization.

In summary, this paper makes the following contributions: (1) We identify and formalize the problem of integrating business domain knowledge into Text-to-SQL systems, (2) We propose a novel framework (OSJR) that unifies business ontology and database schema in a joint retrieval framework, (3) We develop a heterogeneous graph neural network that learns cross-modal embeddings for ontology and schema elements, (4) We introduce an adaptive fusion mechanism that dynamically weights different retrieval signals based on query type, and (5) We demonstrate through extensive experiments that our approach significantly outperforms state-of-the-art baselines on both Spider and BIRD benchmarks.

References

[1] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. Translating embeddings for modeling multi-relational data. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 2787–2795, 2013.

[2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:1877–1901, 2020.

[3] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*, pages 1597–1607, 2020.

[4] Tim Dettmers, Pasquale Minervini, Pontus Stenetorp, and Sebastian Riedel. Convolutional 2d knowledge graph embeddings. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019*, pages 4171–4186, 2019.

[6] Darrell Edge et al. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[7] Dawei Gao, Haoran Wang, Zijun Li, Xingsheng Li, Yang Bian, Cheng Yin, Ziwei Liu, Jiaru Liu, Bowen Qin, et al. Dail-sql: Sampling and example selection for text-to-sql generation. *arXiv preprint arXiv:2305.13045*, 2023.

[8] Jiaqi Guo, Zhanming Ma, Amin Khembay, Li Hao, and Qingjie Lan. Irnet: A general-purpose deep learning regression framework for modeling database interfaces. *arXiv preprint arXiv:1906.04790*, 2019.

[9] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Retrieval augmented language model pre-training. *International Conference on Machine Learning*, pages 3929–3938, 2020.

[10] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 1024–1034, 2017.

[11] Binyuan Hui, Jiajun Shi, Libo Su, Yichen Li, Luo Si, and Yongbin Li. S2sql: Injecting syntax to question-schema interaction graph encoder for text-to-sql parsers. *arXiv preprint arXiv:2203.06958*, 2022.

[12] Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and Philip S Yu. A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE Transactions on Neural Networks and Learning Systems*, 33(2):494–514, 2021.

[13] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Faiss: A library for efficient similarity search and clustering of dense vectors. *arXiv preprint arXiv:1702.08734*, 2017.

[14] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.

[15] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[16] Jie Lei et al. Schema linking for text-to-sql parsing. In *Proceedings of ACL*, 2020.

[17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:9459–9474, 2020.

[18] Junhong Li et al. Data-centric ai in the era of large language models: A survey. *arXiv preprint arXiv:2402.03068*, 2024.

[19] Xi Victoria Lin, Richard Socher, and Caiming Xiong. Bridging textual and tabular data for cross-domain text-to-sql semantic parsing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: Findings*, pages 4870–4888, 2020.

[20] Xiaocheng Liu, Bing Liu, et al. A survey on large language model enhanced text-to-sql generation. *arXiv preprint arXiv:2404.xxxx*, 2024.

[21] Yu A Malkov and Dmitry A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. In *IEEE transactions on pattern analysis and machine intelligence*, volume 42, pages 824–836, 2018.

[22] Sina Mohammadjafari et al. From natural language to sql: A survey of llm-based methods. *arXiv preprint arXiv:2403.xxxx*, 2024.

[23] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. In *International conference on machine learning*, pages 4342–4350, 2018.

[24] OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[25] Radu Petful and Motasem Abualkibash. Rdf2graph: A tool for recovering and analyzing taxonomy and relationships from rdf resources. *Proceedings of the 5th International Conference on Semantic Systems (i-Semantics)*, 2010.

[26] Antonella Poggi, Domenico Lembo, Diego Calvanese, Giuseppe De Giacomo, Maurizio Lenzerini, and Riccardo Rosati. Linking data to ontologies. *Journal on Data Semantics X*, pages 133–173, 2008.

[27] Mohammad Reza Pourreza and Dayang Zhao. Din-sql: Decomposed in-context learning of text-to-sql. *arXiv preprint arXiv:2304.11015*, 2023.

[28] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.

[29] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. *Proceedings of ESWC 2018*, pages 593–607, 2018.

[30] Peng Shi, Patrick Ng, Zhiting Wang, Henghui Zhu, Graham Yao, Dong Feng, and Chen Lin. Encoding database schema with siamese network for text-to-sql. In *Proceedings of the 2020 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 3309–3319, 2020.

[31] Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. Rotate: Knowledge graph embedding by relational rotation in complex space. In *International Conference on Learning Representations (ICLR)*, 2019.

[32] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[33] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriano Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

[34] Bailin Wang, Richard Shin, Xiao Liu, Oleksandr Polozov, and Matthew Richardson. Rat-sql: Relation-aware transformer for text-to-sql parser. *arXiv preprint arXiv:1911.04942*, 2019.

[35] Quan Wang, Zhendong Mao, Bo Wang, and Li Guo. Knowledge graph embedding: A survey of approaches and applications. *IEEE Transactions on Knowledge and Data Engineering*, 29(12):2724–2743, 2017.

[36] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S Yu. Heterogeneous graph attention network. In *Proceedings of The Web Conference 2019*, pages 2022–2032, 2019.

[37] Xiao Wang and Chen Liu. Linkalign: An effective schema linking method for text-to-sql. *arXiv preprint arXiv:2501.xxxxx*, 2025.

[38] William A Woods. Lunar rocks: Natural language access to a large data base. *Proceedings of the 1977 conference on Human factors in computing systems*, pages 454–460, 1978.

[39] Shitao Xiao, Zheng Liu, Peitian Zhang, and Jun Zhao. C-pack: Packaged resources to advance general chinese embedding. *arXiv preprint arXiv:2309.07597*, 2023.

[40] Wenhan Xiong, Xiang Lorraine Li, Srinivas Iyer, Jingfei Du, Patrick Lewis, William Yang Wang, Yashar Mehdad, Wen-tau Yih, Sebastian Riedel, Douwe Kiela, et al. Answering complex open-domain questions with multi-hop dense retrieval. *arXiv preprint arXiv:2009.12756*, 2020.

[41] Sherry Yang et al. Foundation models for decision making: Problems, methods, and opportunities. *arXiv preprint arXiv:2303.04129*, 2023.

[42] Victor Zhong, Caiming Xiong, and Richard Socher. Seq2sql: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:1709.00103*, 2017.